Resume Project Demo

Distributed Task Queue and Job Scheduler

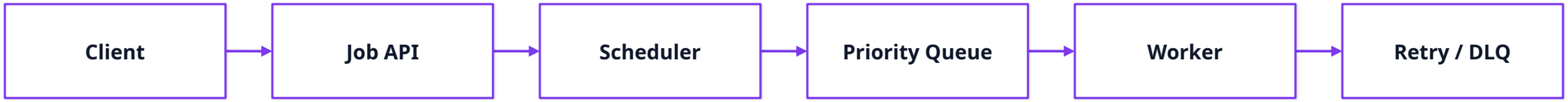
Priority scheduling, worker processing, retry handling and dead-letter design

Built to explain backend engineering, data structures, failure handling, APIs and system design in an SDE I interview.

Demo-ready
Code + Tests + Docker + PDFs

Problem and Architecture

Slow or delayed backend work should not block user requests, and failures must be retried safely.



The design separates request handling, core logic and reliability concerns. That makes it easier to explain scaling and failure handling during interviews.

Key idea

Keep the demo simple enough to run locally, but design it like a real backend service with production concerns.

Core Engineering Features

What this project proves on your resume

- Priority queue
- Scheduled execution
- Fail-first retry demo
- Worker-style processing
- Job event tracking

This project is useful because each feature maps to a real SDE interview topic: data structures, APIs, concurrency, testing, state management, and scale.

Use the feature list to lead the explanation. Do not say only "I built an app". Say what engineering problem each feature solves.

API	**Tests**	**Demo**	**Deploy**
REST	pytest	Browser	Docker

Live Demo Flow

A simple sequence to show the project confidently

1. Create normal job
2. Create fail-first job
3. Run next worker step
4. Show retry then completion

Presenter script

1. Start with the problem.
2. Show the architecture.
3. Run the browser demo.
4. Open the API response.
5. End with what you learned.

Command

```
pip install -r requirements.txt && uvicorn app.main:app --reload --port 8000
```

Interview Talking Points

Use these points to answer project questions clearly

1. Decouples request and work
2. Priority changes execution order
3. Retry handles temporary failure
4. DLQ prevents infinite retries

Resume impact

This project is not just a CRUD app. It shows backend correctness, system design thinking, automated tests and demo ownership.

Best closing line: "I can extend this with Redis, Kafka/SQS, real database migrations, metrics, tracing and AWS deployment."